

Orbital Propagation and Mission Design Using Two-Body Dynamics

AEROSPCE720

Spacecraft Dynamics & Missions 2026

Joshua Anto

860957752

12/04/2026

Table of Contents

Table of Contents	1
List of Figures	2
List of Tables	2
Orbital Propagation	3
Solving Kepler's Equation	3
True Anomaly.....	4
State Conversion	5
Ephemeris Generation and Orbital Evolution	6
Separation Analysis (25 Years).....	8
Numerical Integration with J2 Perturbation	9
Reformulation of Equations of Motion	9
Implementation of TBPwJ2_ECI Function	10
Numerical Integration using ODE45	11
Orbit Propagation Results	11
Specific Energy and Angular Momentum Analysis.....	12
State Conversion: Cartesian State to Orbital Elements (RV to COE)	13
Orbital Element Evolution under J2 Perturbation.....	14
Orbital Maneuvers and Mission design	15
Geostationary Orbit (GEO) insertion	15
Single-Burn Maneuver Design	16
Multi-Burn Strategy under Thrust Constraints	16
Orbital Element Evolution During Insertion	17
Time Evolution of Multi-Burn GEO Insertion	17
Bi-Elliptic Transfer Strategy	18
Phasing Maneuvers	19
Phasing Strategy Selection.....	19
Orbital Period and Longitude Drift Relationship	19
Phasing Orbit Design and Delta-V Requirement	19
Multi-Revolution Phasing Time	20
Optimal Phasing Strategy under Time Constraint	21
Discussion	22
Conclusion.....	22
References.....	23

List of Figures

Figure 1: Kepler's Function.....	4
Figure 2: COE2RV Function.....	6
Figure 3: Ephemeris Function.....	7
Figure 4: Position Magnitude over One Earth Orbit	8
Figure 5: Velocity Magnitude over One Earth Orbit	8
Figure 6: Earth-Phaethon Separation over 25 years	9
Figure 7: TBPwJ2_ECI Function	11
Figure 8:ECI Trajectory with J2 Perturbation over 15 Orbital Periods.....	11
Figure 9: Specific Energy Variation over 15 Orbital Periods	12
Figure 10: Specific Angular Momentum Variation over 15 Orbital Periods	12
Figure 11: RV2COE Function	14
Figure 12: Semi-major variation under J2	15
Figure 13: RAAN variation under J2	15
Figure 14: GEO Insertion at GTO Apogee.....	16
Figure 15: Multi-Burn GEO Insertion	17
Figure 16: Bi-Elliptic Transfer Velocity Diagram at Apogee	18
Figure 17: External Phasing Orbit for GEO Longitude Relocation	19
Figure 18: Figure 18: External Phasing Orbit for GEO Relocation.....	20
Figure 19: Phasing Orbit Size Vs Total Phasing Time	21
Figure 20: Phasing Δv vs Total Phasing Time	21
Figure 21: Main Script	33
Figure 22: Kepler Function	33
Figure 23: COE2RV Function	34
Figure 24: Ephemeris Function.....	35
Figure 25: TBwJ2_ECI Function.....	35
Figure 26: RV2COE Function	36

List of Tables

Table 1: Orbital elements from initial ECI state	14
Table 2: COE during GEO insertion	17
Table 3: Orbital Element and Time Evolution during GEO Insertion	18
Table 4:Phasing Period and Total Relocation Time for Multi-Revolution Strategy	21

Orbital Propagation

Solving Kepler's Equation

Kepler's equation relates the mean anomaly M , eccentric anomaly E , and eccentricity e :

$$M = E - e \sin E$$

Here, M is known from orbital geometry and time evolution, while E must be determined. As E appears both linearly and within a trigonometric term, the equation is transcendental and is solved numerically by reformulating it as

$$f(E) = E - e \sin E - M = 0$$

and applying the Newton–Raphson method. The derivative

$$f'(E) = 1 - e \cos E$$

yields the update

$$E_{n+1} = E_n - \frac{E_n - e \sin E_n - M}{1 - e \cos E_n}$$

An initial guess $E_0 = M$ is used, which is exact for $e = 0$ and remains effective for $0 \leq e < 1$. Over this range, $f'(E) > 0$, ensuring a unique solution and stable convergence.

A MATLAB implementation iterates this update until both the change in E and the residual $f(E)$ satisfy a tolerance of 10^{-12} , ensuring numerical accuracy. The eccentric anomaly is then converted to true anomaly using

$$\tan\left(\frac{\theta}{2}\right) = \sqrt{\frac{1+e}{1-e}} \tan\left(\frac{E}{2}\right),$$

providing the physical angular position along the orbit.

Applying the solver to Phaethon gives

$$\theta = 187.91^\circ$$

placing the asteroid slightly beyond aphelion (180°). In this region, orbital velocity is minimal and the radial distance is largest, so angular motion is slow. Consequently, large time changes produce only small variations in θ , consistent with Kepler's second law. This effect is amplified by Phaethon's high eccentricity, where motion is rapid near perihelion and significantly slower near aphelion.

The Newton–Raphson method converged rapidly within the prescribed tolerance, confirming the robustness and accuracy of the approach.

The Implemented function is shown below:

```

function[theta] = Kepler(e,M,Tol)
% Kepler's Equation Solver using Newton-Raphson
% Inputs:
% e - eccentricity (0 <= e < 1)
% M - mean anomaly (radians)
% Tol - tolerance for convergence
%
% Output:
% theta - true anomaly (radians)

% Initial Guess for E
E = M; % Letting the eccentricity anomaly equal the mean anomaly

% Newton-Raphson iteration
error = 1;
while error > Tol
    f = E - e*sin(E) - M;    % Kepler's equation
    f_prime = 1 - e*cos(E);  % first derivative

    E_new = E - f / f_prime;  % Newton-Raphson update

    error = abs(E_new - E);    % Checking estimation change
    if abs(E_new - E) < Tol && abs(f) < Tol % Ensure convergence by checking both the update step
        (stability) and the residual |f(E)|
        break;
    end
    E = E_new;                % Update Eccentricity anomaly
end
% Convert eccentric anomaly E to true anomaly theta
theta = 2 * atan2( sqrt(1+e)*sin(E/2), sqrt(1-e)*cos(E/2) );
end

```

Figure 1: Kepler's Function

True Anomaly

The true anomaly at t_0 and $t_0 + 100$ days is obtained by propagating the orbit using Keplerian motion. The mean anomaly evolves as

$$M = M_0 + n(t - t_0), n = \sqrt{\frac{\mu}{a^3}},$$

where a heliocentric two-body model is assumed using the Sun's gravitational parameter μ_S . The updated mean anomaly is wrapped using $M = \text{mod}(M, 2\pi)$ to maintain periodicity and numerical stability. Time is expressed in seconds for consistency with μ .

Kepler's equation is then solved numerically for E , and converted to true anomaly θ , giving the mapping $t \rightarrow M \rightarrow E \rightarrow \theta$. The propagation is implemented by computing Δt , updating M , and applying the Kepler solver at each time step.

The evaluated values are

$$\theta(t_0) = 187.9^\circ, \theta(t_0 + 100 \text{ days}) = 202.1^\circ.$$

The increase confirms forward motion, but the relatively small change ($\sim 14^\circ$) indicates slow angular progression. This is consistent with motion near aphelion ($\theta \approx 180^\circ$), where orbital radius is maximal and velocity is minimal. By Kepler's second law, angular motion is reduced in this region.

This effect is amplified by Phaethon's high eccentricity: near aphelion, small time increments produce limited angular change, while much larger variations occur near perihelion. The nonlinear relationship between angular position and distance further reflects the variation in orbital velocity.

The results are consistent numerically and physically, with the solution satisfying Kepler's equation within tolerance and remaining bounded within the expected orbital geometry.

State Conversion

The Cartesian state is obtained by converting the classical orbital elements (COEs) into inertial position and velocity vectors. The transformation is performed in two stages. First, the state is computed in the perifocal (PQW) frame, where the motion is planar:

$$\mathbf{r}_{PQW} = \frac{p}{1 + e \cos \theta} \begin{bmatrix} \cos \theta \\ \sin \theta \\ 0 \end{bmatrix}, \mathbf{v}_{PQW} = \sqrt{\frac{\mu}{p}} \begin{bmatrix} -\sin \theta \\ e + \cos \theta \\ 0 \end{bmatrix},$$

where $p = a(1 - e^2)$. The state is then rotated into the inertial frame using

$$\mathbf{r} = R_3(\Omega)R_1(i)R_3(\omega)\mathbf{r}_{PQW}, \mathbf{v} = R_3(\Omega)R_1(i)R_3(\omega)\mathbf{v}_{PQW}.$$

The angles i , Ω , and ω define the orbital orientation in three-dimensional space. Under the two-body assumption, a , e , i , Ω , and ω remain constant, so the state evolution is governed solely by the true anomaly θ .

The implemented function is shown below.

```
function[X] = COE2RV(oe,mu)
% COE2RV Converts classical orbital elements to inertial position and velocity
%
% Inputs:
% oe = [a; e; i; Omega; w; theta]
%   a - semi-major axis (km)
%   e - eccentricity (-)
%   i - inclination (rad)
%   Omega - RAAN (rad)
%   w - argument of periapsis (rad)
%   theta - true anomaly (rad)
% mu - gravitational parameter of central body (km^3/s^2)
%
% Output:
% X = [x; y; z; vx; vy; vz] in inertial frame

a = oe(1);
e = oe(2);
i = oe(3);
Omega = oe(4);
w = oe(5);
theta = oe(6);

% Semi-latus rectum
p = a * (1 - e^2);

% Position and Velocity in the perifocal frame
r_PQW = (p / (1 + e * cos(theta))) * [cos(theta); sin(theta); 0];
v_PQW = sqrt(mu/p) * [-sin(theta); e + cos(theta); 0];

% Rotation matrix from PQW to inertial frame

Q = [
    cos(Omega)*cos(w) - sin(Omega)*sin(w)*cos(i), -cos(Omega)*sin(w) - sin(Omega)*cos(w)*cos(i),
    sin(Omega)*sin(i);
    sin(Omega)*cos(w) + cos(Omega)*sin(w)*cos(i), -sin(Omega)*sin(w) + cos(Omega)*cos(w)*cos(i), -
    cos(Omega)*sin(i);
    sin(w)*sin(i), cos(w)*sin(i), cos(i)
];

% Converting to inertial frame
r_IJK = Q * r_PQW;
v_IJK = Q * v_PQW;

% State vector
```

```
X = [r_IJK; v_IJK];
end
```

Figure 2: COE2RV Function

The COE2RV function was used to compute the Cartesian state of Phaethon at t_0 and $t_0 + 100$ days. For both cases, the orbital elements were held constant, while θ was updated using the Kepler solver. The state vector is defined as

$$\mathbf{X} = [x \quad y \quad z \quad v_x \quad v_y \quad v_z]^T,$$

with position in km and velocity in km/s.

This implementation separates propagation from coordinate transformation: the Kepler solver updates the orbital position through θ , while COE2RV maps this position into the inertial frame. This modular structure improves clarity and enables straightforward verification.

The computed states are

$$\mathbf{X}(t_0) = \begin{bmatrix} 177588432.03 \\ 275530549.94 \\ 62946707.39 \\ -8.9759 \\ -1.9765 \\ -3.6006 \end{bmatrix}, \mathbf{X}(t_0 + 100 \text{ days}) = \begin{bmatrix} 74635836.65 \\ 211908822.25 \\ 23084403.62 \\ -15.1761 \\ -14.6552 \\ -5.6910 \end{bmatrix}.$$

The position magnitude decreases from $r(t_0) \approx 3.34 \times 10^8$ km

To $r(t_0 + 100 \text{ days}) \approx 2.26 \times 10^8$ km,

indicating motion from the aphelion region toward the Sun. The velocity increases from $v(t_0) \approx 9.87$ km/s

to $v(t_0 + 100 \text{ days}) \approx 21.85$ km/s,

consistent with the vis-viva relation. These trends reflect conservation of orbital energy and the expected behaviour of a highly eccentric orbit, where motion is slow at aphelion and accelerates as the body moves inward.

Ephemeris Generation and Orbital Evolution

The ephemeris function maps time to the Cartesian state of an orbiting body by propagating its classical orbital elements from a reference epoch. Under the two-body assumption, the elements defining orbital size, shape, and orientation remain constant, so the evolution is governed solely by the mean anomaly,

$$M = M_0 + n(t - t_0), n = \sqrt{\frac{\mu}{a^3}}.$$

Since M evolves linearly with time, it provides an efficient basis for propagation. The updated mean anomaly is wrapped within 2π to preserve periodicity and numerical stability. Kepler's equation is then solved to obtain E , which is converted to θ , giving the mapping: $t \rightarrow M \rightarrow E \rightarrow \theta \rightarrow (\mathbf{r}, \mathbf{v})$.

All time quantities are expressed in seconds to maintain consistency with $\mu (\text{km}^3/\text{s}^2)$. The propagation is performed over one Earth orbital period, with time normalised as a percentage of this period to enable direct comparison between Earth and Phaethon.

The implemented function is shown below.

```
function X = Ephemeris(t, ObjData, mu)
% Ephemeris returns inertial position and velocity at time t
%
% Inputs:
%   t       - time in Julian Days
%   ObjData - [t0; a; e; i; Omega; w; M0]
%           t0 : reference epoch (Julian Days)
%           a  : semi-major axis (km)
%           e  : eccentricity (-)
%           i  : inclination (deg)
%           Omega : RAAN (deg)
```

```

%           w      : argument of periapsis (deg)
%           M0      : mean anomaly at t0 (deg)
%   mu       - gravitational parameter of central body (km^3/s^2)
%
% Output:
%   X = [x; y; z; vx; vy; vz] inertial state vector

Tol = 1e-12;

% Extract data
t0 = ObjData(1);
a = ObjData(2);
e = ObjData(3);
i = deg2rad(ObjData(4));
Omega = deg2rad(ObjData(5));
w = deg2rad(ObjData(6));
M0 = deg2rad(ObjData(7));

% Elapsed time in seconds
dt = (t - t0) * 24 * 3600;

% Mean motion
n = sqrt(mu / a^3);

% Propagate mean anomaly
M = M0 + n * dt;
M = mod(M, 2*pi);

% Solve for true anomaly
theta = Kepler(e, M, Tol);

% Updated orbital elements
oe = [a; e; i; Omega; w; theta];

% Convert to inertial state
X = COE2RV(oe, mu);
end

```

Figure 3: Ephemeris Function

The MATLAB function *Ephemeris* takes time, object data, and μ as inputs and returns the state vector

$$\mathbf{X} = [x, y, z, v_x, v_y, v_z]^T.$$

Angular quantities are converted to radians, and elapsed time from the reference epoch is converted from Julian days to seconds. The mean motion is computed, M is updated and wrapped, and the Kepler solver is applied to obtain θ . The updated elements are then passed to COE2RV to compute the inertial state.

The main script applies this function to both Earth and Phaethon over one Earth orbital period using 1000-time steps. At each step, the Cartesian state is evaluated, and the position and velocity magnitudes are computed.

The results show that Earth's position and velocity magnitudes remain nearly constant ($\sim 1.5 \times 10^8$ km and ~ 30 km/s), consistent with its low eccentricity and near-circular orbit, where variations in radius and speed are minimal.

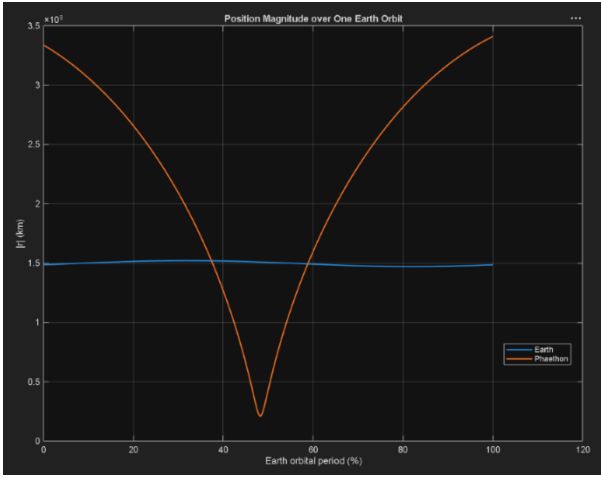


Figure 4: Position Magnitude over One Earth Orbit

In contrast, Phaethon exhibits large variations in both position and velocity, consistent with its highly eccentric orbit. The radius decreases sharply to a minimum near 45–50% of the Earth’s orbital period, corresponding to perihelion, before increasing again toward aphelion.

The velocity shows the inverse behaviour, peaking at over 100 km/s near perihelion and decreasing as the radius increases. This reflects conservation of orbital energy: as the object approaches the Sun, gravitational potential energy is converted into kinetic energy, increasing velocity. This is consistent with the vis-viva relation, $v = \sqrt{\mu \left(\frac{2}{r} - \frac{1}{a} \right)}$,

which shows that velocity increases as orbital radius decreases. The sharp velocity peak highlights the strong non-uniformity of motion in highly eccentric orbits.

This behaviour also illustrates Kepler’s second law. The asteroid spends most of its orbit far from the Sun, where motion is slow and radial variation is gradual, while near perihelion the motion is rapid and confined to a short portion of the orbital period.

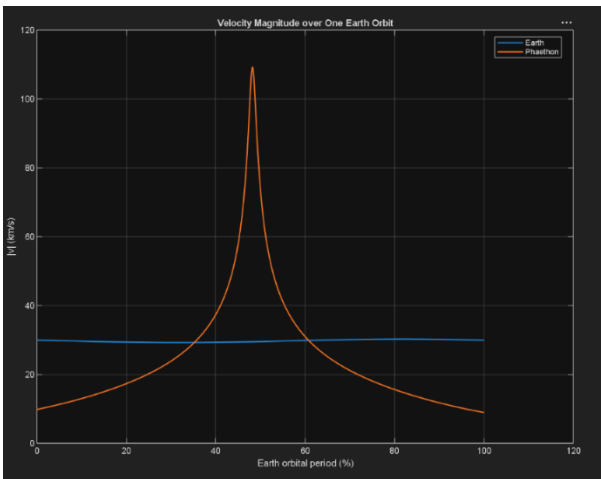


Figure 5: Velocity Magnitude over One Earth Orbit

Overall, the results validate the ephemeris implementation. The near-constant behaviour for Earth confirms correct handling of low-eccentricity motion, while the pronounced variation for Phaethon demonstrates accurate modelling of highly elliptical dynamics.

Separation Analysis (25 Years)

The separation between Earth and Phaethon was evaluated over a 25-year interval using the ephemeris function. At each time step, the heliocentric position vectors were computed and the separation defined as $d(t) = \| \mathbf{r}_{\text{Earth}}(t) - \mathbf{r}_{\text{Phaethon}}(t) \|$.

The propagation was performed over 25 years using 5000-time steps, providing sufficient resolution to capture both long-term trends and close approaches. Time was expressed in years from the reference epoch, and separation in units of 10^6km for clarity. The resulting profile is shown in Figure 6.

The separation exhibits structured but non-uniform behaviour, with sharp minima corresponding to close approaches when both bodies occupy similar heliocentric positions. The spacing between these minima is irregular, indicating quasi-periodic motion. This arises from the difference in orbital periods: as the bodies orbit with different angular velocities, their relative phase continuously shifts, preventing periodic alignment.

Phaethon's high eccentricity further modulates the profile. Its large variation in distance and velocity causes both the timing and depth of minima to vary. The closest approaches occur when alignment coincides with perihelion, while larger separations occur when the bodies are on opposite sides of the Sun or when Phaethon is near aphelion. The separation ranges from below $100 \times 10^6\text{km}$ to above $500 \times 10^6\text{km}$.

Despite the irregular spacing, repeating patterns emerge over longer timescales, indicating an approximate alignment cycle. However, variations in timing and magnitude confirm that the motion is quasi-periodic rather than strictly repeating.

From a mission design perspective, the minima represent potential encounter windows. Their irregular occurrence requires long-term propagation to identify viable opportunities, and feasibility depends on both separation distance and the asteroid's dynamical state at encounter.

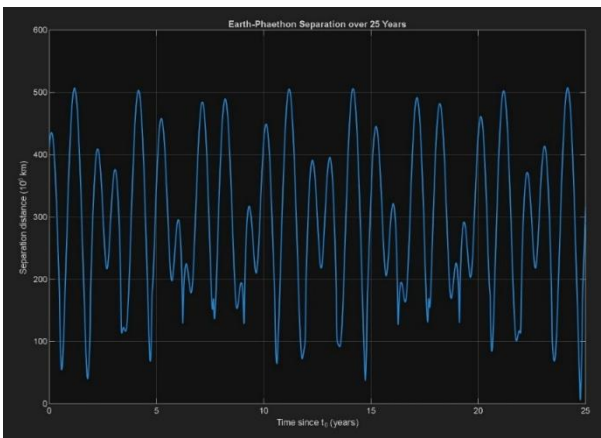


Figure 6: Earth-Phaethon Separation over 25 years

Overall, the results reflect deterministic but non-uniform orbital behaviour consistent with two-body dynamics, providing insight into long-term Earth–asteroid interactions.

Numerical Integration with J2 Perturbation

Reformulation of Equations of Motion

To enable numerical propagation with MATLAB's `ode45`, the second-order equations of motion are reformulated as a first-order system. The two-body dynamics in an Earth-centred inertial frame are $\ddot{\mathbf{r}} = -\frac{\mu}{r^3}\mathbf{r}$, $r = \|\mathbf{r}\|$.

Defining the state vector $\mathbf{X} = [x \ y \ z \ v_x \ v_y \ v_z]^T$,

with $v_x = \dot{x}$, $v_y = \dot{y}$, and $v_z = \dot{z}$, the system becomes

$$\dot{\mathbf{X}} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ a_x \\ a_y \\ a_z \end{bmatrix}.$$

To account for Earth's oblateness, the J_2 perturbation is added to the central acceleration:

$$a_x = -\frac{\mu x}{r^3} + \frac{3\mu J_2 R_E^2}{2r^5} x \left(1 - \frac{5z^2}{r^2}\right),$$

$$a_y = -\frac{\mu y}{r^3} + \frac{3\mu J_2 R_E^2}{2r^5} y \left(1 - \frac{5z^2}{r^2}\right),$$

$$a_z = -\frac{\mu z}{r^3} + \frac{3\mu J_2 R_E^2}{2r^5} z \left(3 - \frac{5z^2}{r^2}\right)$$

The dynamics are thus expressed as a six-dimensional autonomous system $\dot{\mathbf{X}} = f(\mathbf{X})$, suitable for numerical integration.

Physically, the first three components propagate position, while the final three describe acceleration due to central gravity and the J_2 perturbation. The inclusion of J_2 introduces non-radial effects, enabling modelling of phenomena such as nodal precession and short-period variations.

Implementation of TBPwJ2_ECI Function

The first-order system derived in Section 4.1 was implemented in MATLAB using the function *TBPwJ2_ECI*, which returns the time derivative of the ECI state required by *ode45*. The input state vector is $\mathbf{X} = [x \ y \ z \ v_x \ v_y \ v_z]^T$,

with position and velocity components extracted to compute the instantaneous radius r . The central gravitational and J_2 perturbation accelerations are then evaluated and combined to form the total acceleration.

The function returns $\dot{\mathbf{X}} = [v_x \ v_y \ v_z \ a_x \ a_y \ a_z]^T$,

which matches the required first-order system.

```
function f = TBPwJ2_ECI(t, X, mu, J2, RE)
% TBPwJ2_ECI
% Returns the time derivative of the ECI state vector for the
% two-body problem including the J2 perturbation.
%
% Inputs:
% t - time (s) [not used explicitly, but required by ode45]
% X - state vector [x; y; z; vx; vy; vz] in km and km/s
% mu - gravitational parameter of central body (km^3/s^2)
% J2 - second zonal harmonic (-)
% RE - equatorial radius of central body (km)
%
% Output:
% f - state derivative [vx; vy; vz; ax; ay; az]

% Extract position and velocity
x = X(1);
y = X(2);
z = X(3);
vx = X(4);
vy = X(5);
vz = X(6);

% Radius magnitude
r = sqrt(x^2 + y^2 + z^2);

% Two-body acceleration
ax_2B = -mu*x/r^3;
ay_2B = -mu*y/r^3;
az_2B = -mu*z/r^3;

% J2 acceleration
factor_J2 = -(3*mu*J2*RE^2)/(2*r^5);

ax_J2 = factor_J2 * x * (1 - 5*z^2/r^2);
ay_J2 = factor_J2 * y * (1 - 5*z^2/r^2);
az_J2 = factor_J2 * z * (3 - 5*z^2/r^2);

% Total acceleration
ax = ax_2B + ax_J2;
```

```
ay = ay_2B + ay_J2;
az = az_2B + az_J2;
```

```
% State derivative
```

```
f = [vx;
```

```
vy;
```

```
vz;
```

```
ax;
```

```
ay;
```

```
az];
```

```
end
```

Figure 7: TBPwJ2_ECI Function

Although time t is included as an input, the system is autonomous and does not explicitly depend on it. The argument is retained to satisfy the required function form $f(t, \mathbf{X})$ for `ode45`, providing a direct numerical representation of the spacecraft dynamics with J_2 perturbation included.

Numerical Integration using ODE45

Using the initial ECI state from Eq. (11), the motion was propagated with MATLAB's `ode45` using the TBPwJ2_ECI model, which includes central gravity and the J_2 perturbation.

The initial semi-major axis was estimated using the vis-viva relation, $a = \left(\frac{2}{r_0} - \frac{v_0^2}{\mu_E} \right)^{-1}$

Giving $a = 7178.14$ km

The corresponding orbital period is $T = 2\pi \sqrt{\frac{a^3}{\mu_E}} = 6052.42$ s ≈ 1.681 h

With $R_E = 6378.14$ km, this corresponds to an altitude of approximately 800 km, confirming a realistic low Earth orbit.

The system was then propagated over 15 orbital periods with outputs every 60 s. Solver tolerances were set to $\text{RelTol} = 10^{-12}$ and $\text{AbsTol} = 10^{-12}$, ensuring high numerical accuracy.

Orbit Propagation Results

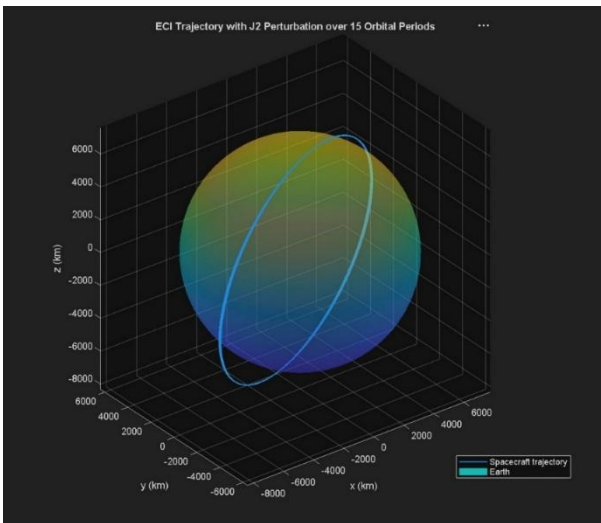


Figure 8: ECI Trajectory with J_2 Perturbation over 15 Orbital Periods

The trajectory corresponds to an inclined low Earth orbit and remains entirely above Earth's surface, confirming that the initial state is non-intersecting. The orbit size is consistent with the computed semi-major axis (~ 7178 km), corresponding to an altitude of approximately 800 km, indicating a realistic LEO.

Over 15 orbital periods, the trajectory does not close, which is expected with the inclusion of the J_2 perturbation. Earth's oblateness introduces non-central effects that produce gradual changes in orbital geometry, including nodal and apsidal precession. The observed drift is therefore a physical consequence of the model rather than a numerical error.

Overall, the propagation is physically consistent, with the orbit remaining bounded, correctly scaled, and exhibiting the expected non-closed behaviour of J_2 -perturbed motion in the ECI frame.

Specific Energy and Angular Momentum Analysis

The propagated ECI states were used to evaluate the spacecraft's specific kinetic energy, potential energy, total mechanical energy, and angular momentum over the 15-orbit integration.

At each time step, the position and velocity magnitudes were computed as $r = \|\mathbf{r}\|$, $v = \|\mathbf{v}\|$.

The specific energies were then evaluated as $\epsilon_K = \frac{1}{2}v^2$, $\epsilon_P = -\frac{\mu_E}{r}$, $\epsilon = \epsilon_K + \epsilon_P$.

The specific angular momentum vector was computed as $\mathbf{h} = \mathbf{r} \times \mathbf{v}$, $h = \|\mathbf{h}\|$.

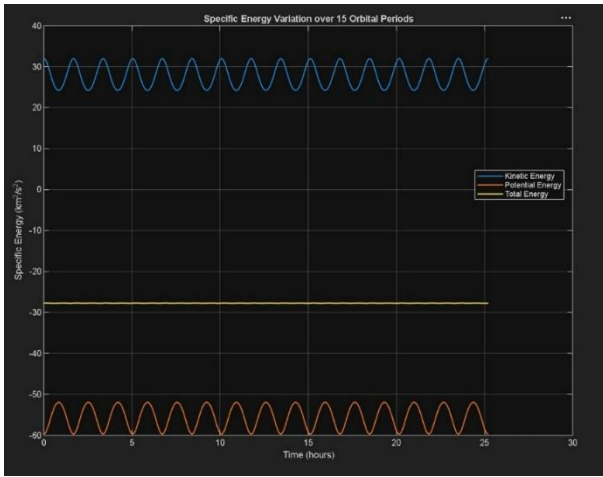


Figure 9: Specific Energy Variation over 15 Orbital Periods

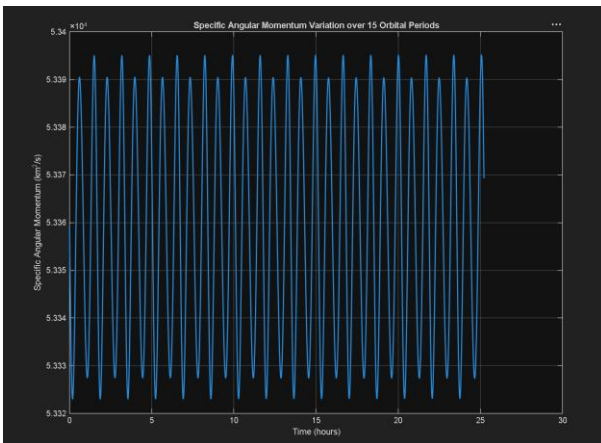


Figure 10: Specific Angular Momentum Variation over 15 Orbital Periods

The specific kinetic and potential energies exhibit periodic oscillations consistent with elliptical motion. As the spacecraft approaches perigee, velocity increases and the kinetic energy rises while the potential energy decreases in magnitude. The opposite behaviour occurs near apogee, resulting in the expected out-of-phase relationship between the two quantities.

The total specific mechanical energy remains approximately constant, with only small, bounded variations. In a purely two-body system this quantity is conserved; the observed deviations arise from the inclusion of the J_2 perturbation while the plotted potential assumes a central gravitational field. The absence of any secular drift confirms that the propagation remains dynamically stable and free from non-conservative effects.

The specific angular momentum magnitude also remains nearly constant, with small periodic variations. Unlike the two-body case, the J_2 perturbation introduces a non-radial component of acceleration, producing a small torque that causes precession of the orbital plane. Consequently, the direction of the angular momentum vector evolves over time, while its magnitude remains effectively conserved.

Overall, the results are physically consistent, capturing the expected exchange between kinetic and potential energy and the perturbation-induced evolution of orbital geometry.

State Conversion: Cartesian State to Orbital Elements (RV to COE)

The initial ECI state vector was converted to classical orbital elements using the *RV2COE* function, providing a physically interpretable description of the orbit in terms of a , e , i , Ω , ω , and θ .

The angular momentum and node vectors are computed as $\mathbf{h} = \mathbf{r} \times \mathbf{v}$, $\mathbf{n} = \mathbf{k} \times \mathbf{h}$,

where $\mathbf{k}$ is the inertial z -axis. The eccentricity vector is $\mathbf{e} = \frac{1}{\mu} \left[\left(v^2 - \frac{\mu}{r} \right) \mathbf{r} - (\mathbf{r} \cdot \mathbf{v}) \mathbf{v} \right]$,

and the specific mechanical energy is $\epsilon = \frac{v^2}{2} - \frac{\mu}{r}$, $a = -\frac{\mu}{2\epsilon}$.

The angular elements are computed using atan2-based expressions and wrapped within $[0, 2\pi]$ to ensure correct quadrant representation. The *RV2COE* function takes the Cartesian state and gravitational parameter as inputs and returns the corresponding orbital elements.

```
function coe = RV2COE(X, mu)
% RV2COE Converts ECI position and velocity to classical orbital elements
%
% Inputs:
% X = [x; y; z; vx; vy; vz] (km, km/s)
% mu = gravitational parameter (km^3/s^2)
%
% Output:
% coe = [a; e; i; Omega; w; theta]
% angles returned in radians

r_vec = X(1:3);
v_vec = X(4:6);

r = norm(r_vec);
v = norm(v_vec);

% Angular momentum
h_vec = cross(r_vec, v_vec);
h = norm(h_vec);

% Node vector
k_hat = [0; 0; 1];
n_vec = cross(k_hat, h_vec);
n = norm(n_vec);

% Eccentricity vector
e_vec = (1/mu) * ((v^2 - mu/r)*r_vec - dot(r_vec, v_vec)*v_vec);
e = norm(e_vec);

% Specific mechanical energy
energy = 0.5*v^2 - mu/r;

% Semi-major axis
a = -mu / (2*energy);

% Inclination
i = acos(h_vec(3)/h);

% RAAN
```

```

Omega = atan2(n_vec(2), n_vec(1));
Omega = mod(Omega, 2*pi);

% Argument of periapsis
w = atan2(dot(cross(n_vec, e_vec), h_vec)/h, dot(n_vec, e_vec));
w = mod(w, 2*pi);

% True anomaly
theta = atan2(dot(cross(e_vec, r_vec), h_vec)/h, dot(e_vec, r_vec));
theta = mod(theta, 2*pi);

% COE vector
coe = [a; e; i; Omega; w; theta];
end

```

Figure 11: RV2COE Function

The function was applied to the initial state vector, with angular elements converted to degrees for reporting. The recovered classical orbital elements are presented in Table 1.

Element	Value
a	7178.137 km
e	0.0700
i	98.518 deg
Ω	20 deg
ω	45 deg
θ	360 deg

Table 1: Orbital elements from initial ECI state

The recovered orbital elements are physically consistent. The semi-major axis corresponds to a low Earth orbit with an altitude of approximately 800 km, and the eccentricity of 0.07 indicates a mildly elliptical trajectory.

The inclination of 98.518° confirms a retrograde, near-polar orbit, which is consistent with the strong J_2 -induced nodal precession observed. The angular elements are well-conditioned, with $\Omega = 20^\circ$ and $\omega = 45^\circ$ matching a consistent orbital configuration. The value $\theta = 360^\circ$ is equivalent to 0° , indicating the spacecraft begins at periapsis due to angle wrapping rather than error.

Overall, the RV2COE conversion yields a physically meaningful orbit and provides validation of the propagated state.

Orbital Element Evolution under J2 Perturbation

The evolution of the semi-major axis and RAAN was obtained by converting the propagated ECI states into classical orbital elements at each time step using the RV2COE function, applied at 60-second intervals over the 15-orbit integration.

The resulting histories were plotted as functions of time. To capture secular behaviour, the RAAN was unwrapped prior to conversion to degrees, ensuring a continuous representation rather than a 2π discontinuity.

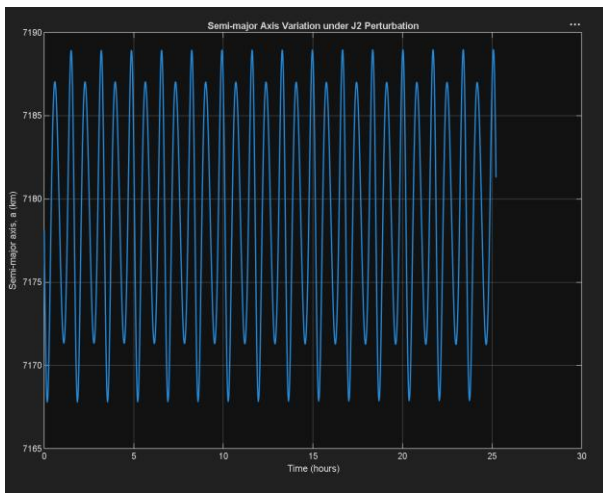


Figure 12: Semi-major variation under J_2

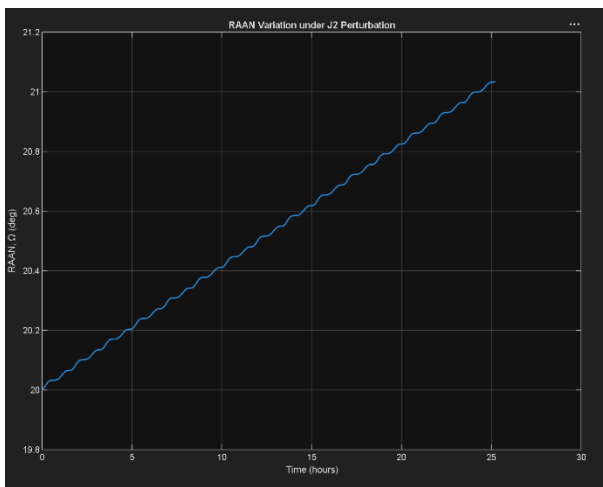


Figure 13: RAAN variation under J_2

The semi-major axis remains effectively constant throughout the propagation, with only small, short-period oscillations. The orbital energy is conserved, and that the J_2 perturbation does not introduce any secular change in the orbit size.

In contrast, the right ascension of the ascending node exhibits a clear secular variation. The RAAN increases from 20° to approximately 20.998° over one day, corresponding to a drift of about $+0.998^\circ/\text{day}$. This nearly linear trend, with small, superimposed oscillations, is characteristic of nodal precession induced by Earth's oblateness.

This behaviour arises because the J_2 perturbation introduces a non-central component of acceleration, producing a torque on the orbital plane. As a result, the orbital plane rotates gradually about the Earth's axis while maintaining its inclination.

The magnitude of the observed precession rate is close to the Earth's mean motion around the Sun ($\sim 0.9856^\circ/\text{day}$), indicating that the orbit maintains a nearly constant orientation relative to the Sun. The trajectory is a near Sun-synchronous orbit.

Orbital Maneuvers and Mission design

Geostationary Orbit (GEO) insertion

The insertion of a spacecraft into geostationary orbit (GEO) requires simultaneous control of orbital energy and inclination. Starting from a geostationary transfer orbit (GTO), the spacecraft must be circularised at the GEO radius while removing the launch-induced inclination.

This section analyses GEO insertion under practical constraints. An ideal impulsive manoeuvre is first used to illustrate the combined requirements of circularisation and plane change. This is then extended to a multi-burn strategy constrained by thrust and burn duration. Finally, a bi-elliptic transfer is evaluated to assess its effectiveness for inclination reduction and its overall feasibility.

Single-Burn Maneuver Design

The transfer from GTO to GEO is achieved by an impulsive manoeuvre at apogee, where the spacecraft is at $(r_{\text{GEO}}, 0, 0)$ with purely tangential velocity. However, the initial GTO is inclined at 28.5° , so the velocity has an out-of-plane component. GEO insertion must therefore both circularise the orbit and remove this inclination.

The required impulse is $\Delta \mathbf{v} = \mathbf{v}_{\text{GEO}} - \mathbf{v}_{\text{GTO}}$,

with an along-track component to match the circular GEO velocity and a normal component to eliminate the out-of-plane motion.

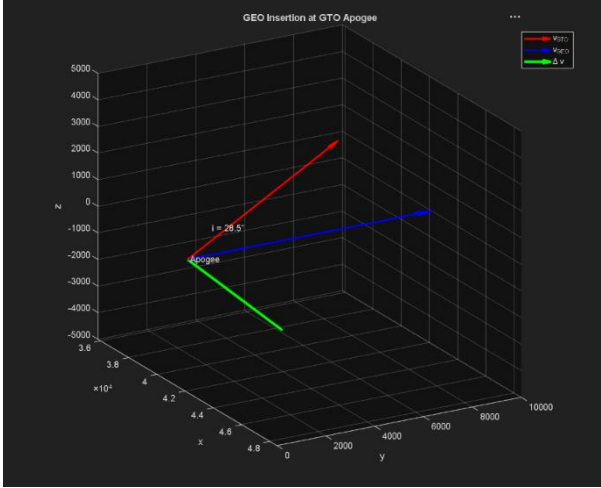


Figure 14: GEO Insertion at GTO Apogee

Figure 14 illustrates the manoeuvre geometry in the ECI frame. The initial velocity $\mathbf{v}_{\text{GTO}}$ is inclined, while $\mathbf{v}_{\text{GEO}}$ lies in the equatorial plane. Removing the out-of-plane component associated with 28.5° produces a velocity change with both tangential and normal components, indicating that circularisation and inclination removal are performed simultaneously.

Multi-Burn Strategy under Thrust Constraints

The GEO insertion requires a total velocity increment of approximately 1.8 km/s, which cannot be achieved in a single burn due to propulsion limits. With thrust $F = 1400$ N and maximum burn duration $t_{\text{thr}} = 400$ s, the maximum increment per burn is $\Delta v_{\text{max}} = \frac{F}{m_{\text{sat}}} t_{\text{thr}} = 0.56$ km/s

Since $\Delta v_{\text{max}} \ll \Delta v$, the manoeuvre must be distributed across multiple burns applied at successive apogees and aligned with the total correction, $\Delta \mathbf{v} = \sum_{n=1}^{N_{\text{man}}} \Delta \mathbf{v}_n$

The minimum number of burns is $N_{\text{man}} = \left\lceil \frac{\Delta v}{\Delta v_{\text{max}}} \right\rceil = 4$

with the ceiling ensuring the total increment meets the required Δv . This reflects the practical constraint of finite thrust and burn duration, requiring a staged rather than impulsive manoeuvre.

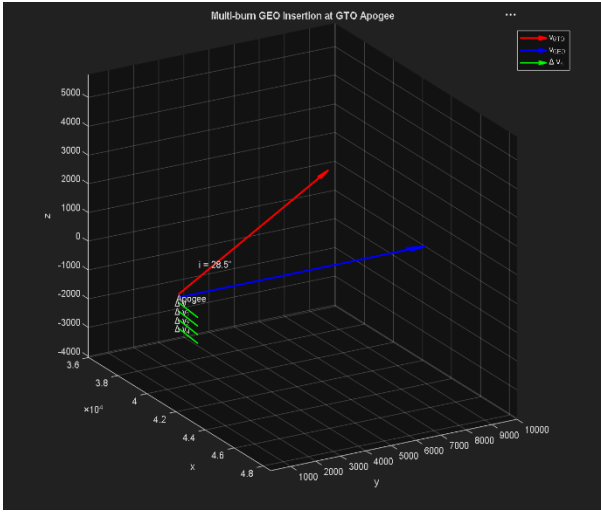


Figure 15: Multi-Burn GEO Insertion

Figure 15 illustrates the subdivision of the manoeuvre into four equal impulses applied at successive apogee passages. Each burn is aligned with the total correction, producing a cumulative increase in orbital energy while progressively reducing inclination, resulting in a stepwise transition to GEO.

Applying the burns at apogee is advantageous because the orbital velocity is minimal, reducing the cost of the inclination change.

Orbital Element Evolution During Insertion

The evolution of the orbit during the multi-burn GEO insertion was analysed by converting the state vector to classical orbital elements after each burn. Starting from the GTO apogee state, successive velocity increments were applied in the direction of the total correction, and the updated state was converted using the *RV2COE* function.

The resulting orbital elements for $n = 0$ to $n = N_{\text{man}}$ are presented in Table 2.

Burn(n)	a (km)	e	i (deg)	Ω (deg)	ω (deg)	θ (deg)
0	24371.07	0.73	28.5	0	180	180
1	26116.53	0.61	17.4	0	180	180
2	28993.97	0.45	9.7	0	180	180
3	33747.01	0.25	4.1	0	180	180
4	42164	1.88e-16	0	180	0	180

Table 2: COE during GEO insertion

The results show a clear transition from the initial inclined, elliptical GTO to a circular equatorial GEO orbit. The semi-major axis increases monotonically with each burn, approaching the GEO radius as orbital energy is added. At the same time, the eccentricity decreases, indicating progressive circularisation.

The inclination is reduced at each step, reflecting the removal of the out-of-plane velocity component through the normal component of the applied Δv . By the final burn, the inclination reaches zero, confirming alignment with the Earth's equatorial plane.

The RAAN remains constant throughout the manoeuvre, as all burns are applied at the same spatial location and do not introduce rotation of the orbital plane about the Earth's axis. Variations in the argument of perigee and true anomaly are observed due to the changing orbital geometry, but these become less meaningful as the orbit approaches a circular configuration.

Overall, the orbital element evolution confirms that the multi-burn strategy successfully reproduces the combined circularisation and inclination removal of the ideal impulsive manoeuvre while satisfying the imposed thrust constraints.

Time Evolution of Multi-Burn GEO Insertion

The time required for the multi-burn GEO insertion was determined from the orbital period of each intermediate orbit using Kepler's

third law, $T = 2\pi \sqrt{\frac{a^3}{\mu}}$

As the semi-major axis increases after each burn, the orbital period correspondingly increases, approaching the geostationary value of approximately 24 hours.

Burn(n)	a (km)	e	i (deg)	Ω (deg)	ω (deg)	θ (deg)	T (hours)	T (days)
0	24371.07	0.73	28.5	0	180	180	10.52	0.44
1	26116.53	0.61	17.4	0	180	180	11.67	0.48
2	28993.97	0.45	9.7	0	180	180	13.65	0.57
3	33747.01	0.25	4.1	0	180	180	17.14	0.71
4	42164	1.88e-16	0	180	0	180	23.93	0.99

Table 3: Orbital Element and Time Evolution during GEO Insertion

Table 3 shows the orbital periods increasing from approximately 10.5 hours to 23.9 hours as the spacecraft gains energy and approaches GEO.

The total transfer time is the sum of the intermediate orbital periods. Since each burn occurs at apogee after one revolution, this corresponds to the cumulative duration of the transfer orbits, excluding the final GEO orbit. The resulting transfer time is approximately 2.21 days.

Bi-Elliptic Transfer Strategy

An alternative GEO insertion strategy is the bi-elliptic transfer, consisting of three impulsive manoeuvres. A first burn at GTO perigee raises the apogee well beyond GEO without changing inclination. At this high apogee, a second burn reduces inclination while raising the perigee toward GEO. After half an orbit, a final burn at GEO radius circularises the orbit.

The key advantage lies in the second manoeuvre, performed at high apogee where orbital velocity is lower, reducing the cost of inclination change. The corresponding geometry is shown in Figure 16.

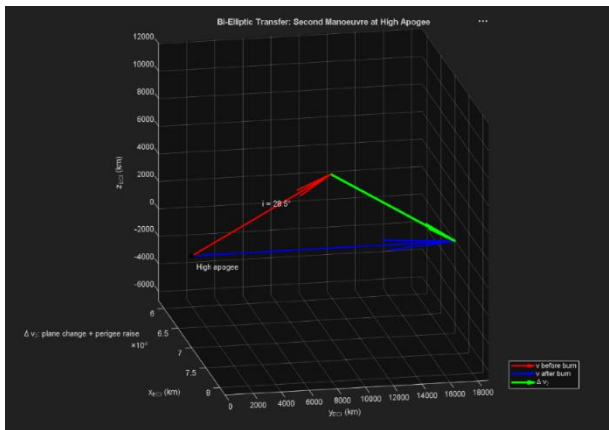


Figure 16: Bi-Elliptic Transfer Velocity Diagram at Apogee

In Figure 16, the initial velocity $\mathbf{v}_{\text{before}}$ is inclined relative to the equatorial plane, while $\mathbf{v}_{\text{after}}$ lies closer to it and has a greater magnitude due to increased orbital energy. The impulse $\Delta \mathbf{v}_2$ removes the out-of-plane component while raising the perigee, combining inclination change and energy increase in a single burn.

This advantage is offset by the additional energy required to raise the apogee above GEO and return to GEO altitude. For an initial inclination of 28.5° , the total Δv is not significantly reduced compared to direct insertion.

The approach is also incompatible with the thrust constraint. The required burns, particularly the initial apogee-raising manoeuvre, exceed the allowable Δv per burn, requiring further subdivision and increasing complexity. The transfer time is also longer due to the large period of the high-apogee orbit.

Although bi-elliptic transfers can reduce inclination change cost for large angles, they are neither the most efficient nor practical option here compared to the multi-burn GEO insertion strategy.

Phasing Maneuvers

Phasing manoeuvres adjust a spacecraft's angular position without permanently altering the orbit. In GEO, this is achieved by temporarily changing the orbital period through a phasing orbit with a different semi-major axis, allowing the spacecraft to advance or lag relative to Earth's rotation before returning to GEO at the target longitude.

Phasing Strategy Selection

The required relocation is $\Delta\lambda = -80^\circ$, so the spacecraft must fall behind its current GEO longitude, requiring a reduced angular rate.

An external phasing orbit is therefore selected. With $a_{ph} > r_{GEO}$, the orbital period increases, causing westward drift relative to Earth's rotation and accumulating the required phase lag. An internal orbit would produce eastward drift and is therefore unsuitable.

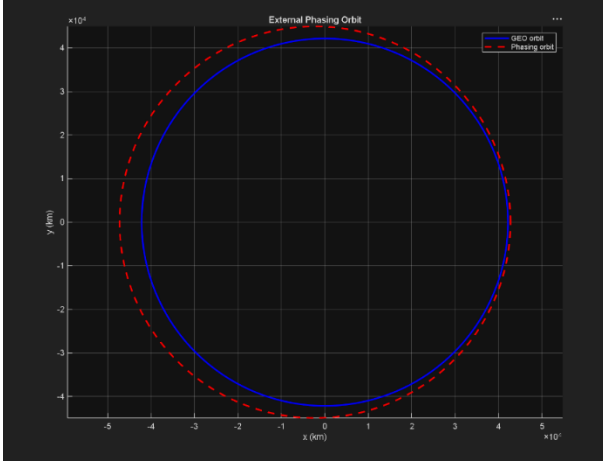


Figure 17: External Phasing Orbit for GEO Longitude Relocation

Figure 17 shows the GEO orbit and the external phasing orbit. The larger semi-major axis produces a longer period, allowing the spacecraft to lag before a final tangential burn returns it to GEO at the target longitude.

Orbital Period and Longitude Drift Relationship

The required relocation is $\Delta\lambda = -80^\circ$, so the spacecraft must lag its initial GEO position. This is achieved by introducing a period difference relative to GEO. The corresponding time offset is $\Delta t_{ph} = \frac{\Delta\lambda}{360^\circ} T_{GEO}$.

With $T_{GEO} = 86164$ s, $\Delta t_{ph} = -19147.6$ s.

Since $\Delta t_{ph} = T_{GEO} - T_{ph}$,

a negative value implies $T_{ph} > T_{GEO}$. The phasing period is therefore, $T_{ph} = 105311.6$ s ≈ 29.25 h.

A longer period and lower angular velocity than GEO, producing westward drift and the required phase lag, consistent with an external phasing orbit.

Phasing Orbit Design and Delta-V Requirement

Using the phasing period, the semi-major axis is obtained from Kepler's third law, $a_{ph} = \left[\mu_E \left(\frac{T_{ph}}{2\pi} \right)^2 \right]^{1/3}$.

For $T_{ph} = 105311.6$ s, $a_{ph} = 48255$ km.

Since $a_{ph} > r_{GEO}$, the orbit is external. The orbit is tangent to GEO at the burn point, giving

$$\begin{aligned} r_p &= r_{GEO} = 42164 \text{ km}, \\ r_a &= 2a_{ph} - r_p = 54346 \text{ km}, \end{aligned}$$

$$e_{ph} = \frac{r_a - r_p}{r_a + r_p} = 0.126.$$

Figure 18 shows the GEO orbit and the external phasing orbit. The larger semi-major axis produces a longer period, allowing the spacecraft to lag before returning to GEO at the target longitude.

The manoeuvre consists of two tangential burns: the first inserts the spacecraft into the phasing orbit, and the second circularises it at GEO after one revolution.

The required velocity change is $\Delta v_{ph} = 2 |v_{ph,p} - v_{GEO}| \approx 0.37$ km/s.

This relatively small Δv reflects that the manoeuvre primarily adjusts orbital timing rather than energy, making phasing highly efficient for GEO longitude relocation.

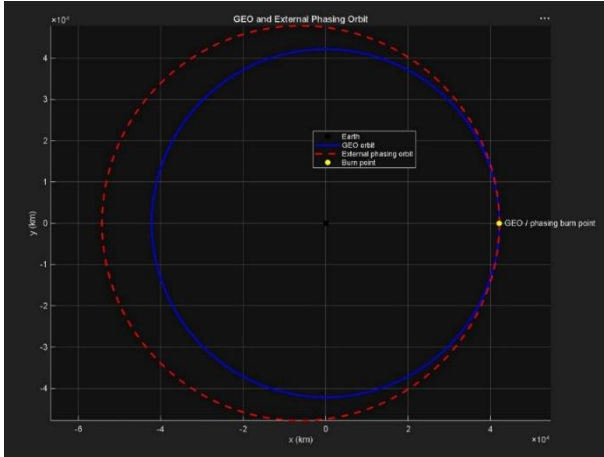


Figure 18: Figure 18: External Phasing Orbit for GEO Relocation

Multi-Revolution Phasing Time

The longitude relocation can be distributed over multiple phasing revolutions rather than achieved in a single orbit. While the total phase change remains $\Delta\lambda = -80^\circ$, the required timing offset is divided across N_{ph} revolutions.

The time offset per revolution is $\Delta t_{ph,N} = \frac{1}{N_{ph}} \frac{\Delta\lambda}{360^\circ} T_{GEO}$

Using $\Delta t_{ph,N} = T_{GEO} - T_{ph,N}$

the corresponding phasing period is $T_{ph,N} = T_{GEO} - \Delta t_{ph,N}$

The total phasing time is then, $t_{ph,N} = N_{ph} T_{ph,N}$

Nph	Tph (s)	Tph (hrs)	Total Time (days)
1	105311.55	29.25	1.22
2	95737.77	26.59	2.21
3	92546.52	25.70	3.21
4	90950.88	25.26	4.21
5	89993.51	24.99	5.20
6	89355.26	24.82	6.20
7	88899.36	24.69	7.20
8	88557.44	24.59	8.19
9	88291.51	24.52	9.19
10	88078.75	24.46	10.19
11	87904.68	24.42	11.19
12	87759.63	24.37	12.18
13	87636.88	24.34	13.18
14	87531.68	24.31	14.18
15	87440.50	24.28	15.18
16	87360.72	24.26	16.17

17	87290.32	24.24	17.17
18	87227.7	24.23	18.17
19	87171.76	24.21	19.16
20	87121.38	24.20	20.16

Table 4: Phasing Period and Total Relocation Time for Multi-Revolution Strategy

Table 4 shows that as N_{ph} increases, the phasing period approaches T_{GEO} , so the orbit becomes closer to GEO. This reduces the required change in semi-major axis and lowers the Δv_{per} manoeuvre. However, more revolutions are required to achieve the same phase shift, increasing transfer time. For example, increasing from $N_{ph} = 1$ to 5 reduces the period difference but increases the relocation time from ~ 1.22 days to over 5.21 days.

This highlights the trade-off: single-revolution phasing minimises time but requires larger Δv , while multi-revolution phasing reduces Δv at the cost of longer duration.

Optimal Phasing Strategy under Time Constraint

The multi-revolution phasing cases were extended by computing the semi-major axis and two-burn velocity cost for each $N_{ph} = 1$ to 20.

The phasing period was converted to semi-major axis using, $a_{ph,N} = \left[\mu_E \left(\frac{T_{ph,N}}{2\pi} \right)^2 \right]^{1/3}$

Since $\Delta\lambda < 0$, all solutions correspond to external orbits with $a_{ph,N} > r_{GEO}$. The total velocity cost was then evaluated as

$$\Delta v_{ph,N} = 2 \mid v_{ph,p,N} - v_{GEO} \mid$$

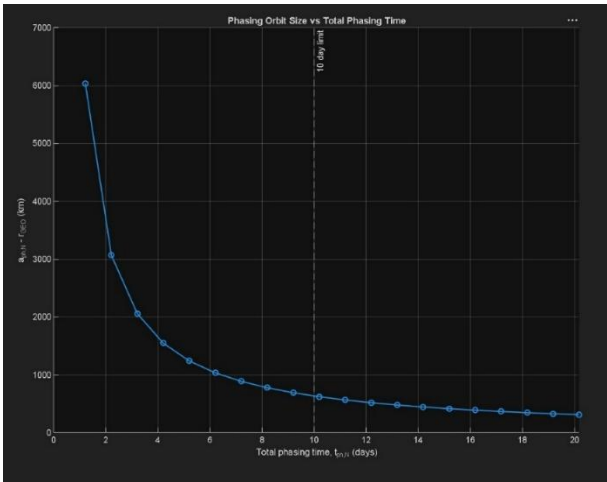


Figure 19: Phasing Orbit Size Vs Total Phasing Time

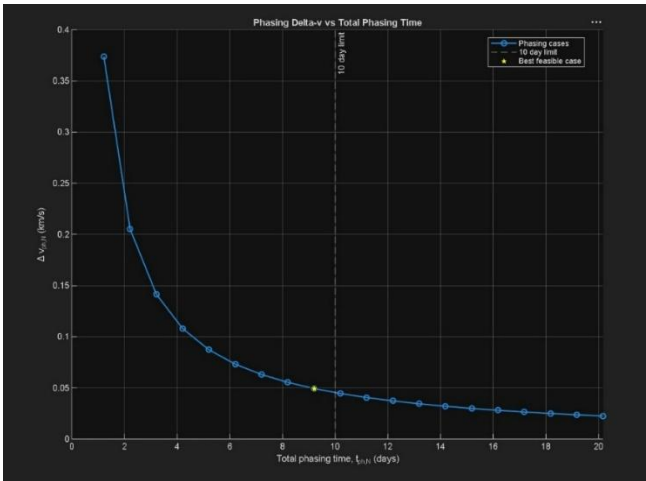


Figure 20: Phasing Δv vs Total Phasing Time

The results show that increasing the number of phasing revolutions reduces the required deviation from GEO. As N_{ph} increases, the phasing period approaches T_{GEO} , causing the semi-major axis to converge toward the GEO radius. Consequently, the required Δv decreases, as the orbit remains closer to circular GEO conditions.

This behaviour reflects a fundamental trade-off between manoeuvre cost and transfer time. Short-duration solutions require larger deviations from GEO and therefore higher Δv , while longer-duration solutions allow smaller orbital adjustments at reduced propellant cost.

Applying the constraint $t_{ph} \leq 10$ days, the optimal solution is obtained at $N_{ph} = 9$, yielding $\Delta v \approx 0.049$ km/s with a transfer time of approximately 9.2 days. This represents the minimum Δv solution that satisfies the operational time limit.

Further increases in N_{ph} would continue to reduce Δv but would violate the time constraint. Therefore, the optimal strategy is to maximise the number of phasing revolutions within the allowable duration.

From an engineering perspective, this result highlights that longitude relocation in GEO is most efficiently achieved by exploiting small differences in orbital period rather than large energy changes, provided sufficient time is available.

Discussion

The analytical propagation, numerical integration, and manoeuvre design results provide a consistent and physically coherent description of orbital motion under both ideal and perturbed conditions. The ephemeris approach efficiently models Keplerian motion with constant orbital elements, as seen in the smooth behaviour of Earth and Phaethon, but neglects perturbations. In contrast, numerical integration incorporating J_2 captures both short-period variations and secular effects, providing a more realistic representation of Earth-orbiting trajectories.

The impact of J_2 is clearly observed. While the semi-major axis remains effectively constant, confirming conservation of orbital energy, the RAAN exhibits steady secular drift with superimposed oscillations due to Earth's oblateness. The magnitude of this precession aligns with theoretical expectations and enables identification of a near Sun-synchronous orbit, demonstrating how small perturbations drive significant long-term changes.

The manoeuvre design results highlight key engineering trade-offs. For GEO insertion, thrust limitations necessitate a multi-burn strategy, increasing transfer time relative to the ideal impulsive case. Phasing manoeuvres similarly trade time for efficiency, with multi-revolution solutions reducing Δv by remaining closer to GEO. The optimal strategy therefore maximises the number of revolutions within the allowed time.

The analysis is based on simplified models. The two-body framework with J_2 neglects higher-order perturbations, and impulsive burns ignore finite thrust and mass variation. While appropriate here, these limit direct application to operational missions.

Overall, the strong agreement between analytical, numerical, and physical interpretations validates the methods, while highlighting the importance of perturbations and trade-offs in realistic mission design.

Conclusion

This work has analysed orbital motion using both analytical and numerical methods and applied these to practical mission design problems. Under the two-body assumption, ephemeris propagation provided an efficient description of Keplerian motion, with Earth and Phaethon exhibiting the expected relationship between orbital radius and velocity. Numerical integration using *ode45* extended this by incorporating the J_2 perturbation, capturing both short-period variations and secular effects. The resulting nodal precession confirmed the influence of Earth's oblateness and enabled identification of a near Sun-synchronous orbit.

The energy and angular momentum results further validate the propagation. Both remain bounded with no long-term drift, indicating a dynamically consistent solution despite the inclusion of perturbations. Conversion between Cartesian states and classical orbital elements provides an additional check, confirming that the expected orbital characteristics are preserved.

The mission design tasks demonstrate the transition from theory to application. For GEO insertion, the ideal impulsive solution minimises Δv , but practical thrust limits require a multi-burn strategy, increasing transfer time while maintaining feasibility. Phasing manoeuvres show that longitude relocation can be achieved efficiently through controlled changes in orbital period, with multi-revolution strategies reducing Δv at the expense of longer transfer times. The optimal solution reflects this trade-off by maximising the number of revolutions within the allowed duration.

Overall, the results are consistent with orbital mechanics theory and show strong agreement between analytical predictions, numerical results, and physical interpretation. While based on simplified models, the analysis provides a solid foundation for more advanced mission design and highlights the importance of including perturbations and operational constraints in realistic scenarios.

References

- Curtis, H. D. (2019). Orbital maneuvers. *Elsevier EBooks*, 287–350. <https://doi.org/10.1016/b978-0-08-102133-0.00006-4>
- Geostationary transfer orbit*. (2024, April 13). Wikipedia. https://en.wikipedia.org/wiki/Geostationary_transfer_orbit
- Orbit Propagation Methods - MATLAB & Simulink*. (2026). Mathworks.com. <https://au.mathworks.com/help/aerotbx/ug/orbit-pop-algorithms.html?requestedDomain=>
- Orbital maneuver*. (2021, April 21). Wikipedia. https://en.wikipedia.org/wiki/Orbital_maneuver
- Phasing Maneuvers — Orbital Mechanics & Astrodynamics*. (n.d.). Orbital-Mechanics.space. <https://orbital-mechanics.space/orbital-maneuvers/phasing-maneuvers.html>
- Section 4.4 - Types of Orbits and Orbital Maneuvers*. (n.d.). Astronomical Returns. <https://www.astronomicalreturns.com/p/section-44-types-of-orbits-and-orbital.html>
- The Newton-Raphson Method*. (n.d.). <https://personal.math.ubc.ca/~anstee/math104/newtonmethod.pdf>
- Two-body problem*. (2020, August 26). Wikipedia. https://en.wikipedia.org/wiki/Two-body_problem
- Wikipedia. (2020, October 25). *Geostationary orbit*. Wikipedia. https://en.wikipedia.org/wiki/Geostationary_orbit
- Wikipedia Contributors. (2019, January 22). *Numerical methods for ordinary differential equations*. Wikipedia; Wikimedia Foundation. https://en.wikipedia.org/wiki/Numerical_methods_for_ordinary_differential_equations
- Wikipedia Contributors. (2022, February 19). *Kepler's equation*. Wikipedia; Wikimedia Foundation. https://en.wikipedia.org/wiki/Kepler%27s_equation
- Wikipedia Contributors. (2024, July 4). *Orbit phasing*. Wikipedia; Wikimedia Foundation.

Appendix

```
clear all;
close all;
clc;
format longG;
delete('*~');

% Parameters
Sun_mu = 1.3271244*10^11; % Sun gravitational parameter in km^3/s^2
Earth_mu = 3.986*10^5; % Earth gravitational parameter in km^3/s^2
radius_Earth = 6378.14; % Radius of earth in km
J2_Earth = 1082.63*10^-6; % J2 perturbation of Earth
radius_GEO = 42164; % radius of GEO in km
period_GEO = 86164; % period of GEO in s
t0 = 2461109.5; % the reference time when we know the orbital elements are accurate in Julian Days for Earth and Phaethon
Tol = 1e-12; % Set tolerance for convergence
```

```

% Earth: OBJData 1
a_Earth = 1.495631294860702*10^8; % Semi-major axis in km
e_Earth = 1.689740748093123*10^-2;

i_Earth = 3.623130579446905*10^-3; % Inclination in degrees
i_Earth = deg2rad(i_Earth); % Inclination in radians

Omega_Earth = 1.716927644253177*10^2; % Right ascension of ascending node in degrees
Omega_Earth = deg2rad(Omega_Earth); % Right ascension of ascending node in radians

w_Earth = 2.900905825693052*10^2; % Argument of periapsis in degrees
w_Earth = deg2rad(w_Earth); % Argument of periapsis in radians

M_Earth = 6.549560435707097*10^1; % Mean anomaly in degrees
M_Earth = deg2rad(M_Earth); % Mean anomaly in radians

% Phaethon: OBJData 2
a_Phaethon = 1.902026305591835*10^8; % Semi-major axis in km
e_Phaethon = 8.896874416008318*10^-1; % Eccentricity

i_Phaethon = 2.231145822457554*10^1; % Inclination in degrees
i_Phaethon = deg2rad(i_Phaethon); % Inclination in radians

Omega_Phaethon = 2.650976504520880*10^2; % Right ascension of ascending node in degrees
Omega_Phaethon = deg2rad(Omega_Phaethon); % Right ascension of ascending node in radians

w_Phaethon = 3.223031004033787*10^2; % Argument of periapsis in degrees
w_Phaethon = deg2rad(w_Phaethon); % Argument of periapsis in radians

M_Phaethon = 2.389210345698454*10^2; % Mean anomaly in degrees
M_Phaethon = deg2rad(M_Phaethon); % Mean anomaly in radians

%% Orbital Propagation 1
%% Task 1.1.1: Find Eccentric anomaly and return True Anomaly

% For Phaethon
theta_Phaethon = Kepler(e_Phaethon, M_Phaethon, Tol);
theta_Phaethon_deg = rad2deg(theta_Phaethon);
disp('Phaethon True Anomaly (degrees):')
disp(theta_Phaethon_deg)

%% Task 1.1.2: Compute true anomaly at t0 and t0 + 100 days for ObjData 2

delta_t_days = 100;
delta_t = delta_t_days*24*3600; % convert days to seconds

% Mean motion of Phaethon about the Sun
n_Phaethon = sqrt(Sun_mu / a_Phaethon^3); % Mean motion in rad/s

% At t0
theta_Phaethon_t0_deg = theta_Phaethon_deg;

% At t0 + 100 days
M_Phaethon_100 = M_Phaethon + n_Phaethon * delta_t;
M_Phaethon_100 = mod(M_Phaethon_100, 2*pi);

theta_Phaethon_100 = Kepler(e_Phaethon, M_Phaethon_100, Tol);
theta_Phaethon_100_deg = rad2deg(theta_Phaethon_100);

disp('Phaethon true anomaly at t0 (degrees):')
disp(theta_Phaethon_t0_deg)

disp('Phaethon true anomaly at t0 + 100 days (degrees):')
disp(theta_Phaethon_100_deg)

%% Task 1.1.3: COE to RV for Phaethon at t0 and t0 + 100 days

% At t0
oe_Phaethon_t0 = [a_Phaethon;

```

```

    e_Phaethon;
    i_Phaethon;
    Omega_Phaethon;
    w_Phaethon;
    theta_Phaethon];
X_Phaethon_t0 = COE2RV(oe_Phaethon_t0, Sun_mu);

% At T0 + 100 days
oe_Phaethon_100 = [a_Phaethon;
    e_Phaethon;
    i_Phaethon;
    Omega_Phaethon;
    w_Phaethon;
    theta_Phaethon_100];

X_Phaethon_100 = COE2RV(oe_Phaethon_100, Sun_mu);

disp('Phaethon state at t0 [x y z vx vy vz]:')
disp(X_Phaethon_t0)

disp('Phaethon state at t0 + 100 days [x y z vx vy vz]:')
disp(X_Phaethon_100)

%% Task 1.1.4 - Ephemeris and state magnitude plots

% Object data: [t0; a; e; i; RAAN; omega; M] with angles in degrees
ObjData1 = [t0; a_Earth; e_Earth; rad2deg(i_Earth); ...
    rad2deg(Omega_Earth); rad2deg(w_Earth); rad2deg(M_Earth)];

ObjData2 = [t0; a_Phaethon; e_Phaethon; rad2deg(i_Phaethon); ...
    rad2deg(Omega_Phaethon); rad2deg(w_Phaethon); rad2deg(M_Phaethon)];

% Earth orbital period
T_Earth = 2*pi*sqrt(a_Earth^3/Sun_mu); % [s]
T_Earth_days = T_Earth/86400; % [days]

% Time vector over one Earth orbit
N = 1000;
t_vec = linspace(t0, t0 + T_Earth_days, N);
period_percent = 100*(t_vec - t0)/T_Earth_days;

% Preallocate state magnitude arrays
rmag_Earth = zeros(1, N);
vmag_Earth = zeros(1, N);
rmag_Phaethon = zeros(1, N);
vmag_Phaethon = zeros(1, N);

% Propagate both objects
for k = 1:N
    X_Earth = Ephemeris(t_vec(k), ObjData1, Sun_mu);
    X_Phaethon = Ephemeris(t_vec(k), ObjData2, Sun_mu);

    rmag_Earth(k) = norm(X_Earth(1:3));
    vmag_Earth(k) = norm(X_Earth(4:6));
    rmag_Phaethon(k) = norm(X_Phaethon(1:3));
    vmag_Phaethon(k) = norm(X_Phaethon(4:6));
end

% Position magnitude
figure;
plot(period_percent, rmag_Earth, 'LineWidth', 1.5); hold on;
plot(period_percent, rmag_Phaethon, 'LineWidth', 1.5);
grid on;
xlabel('Earth orbital period (%)');
ylabel('|r| (km)');
title('Position Magnitude over One Earth Orbit');
legend('Earth', 'Phaethon', 'Location', 'best');

% Velocity magnitude

```

```

figure;
plot(period_percent, vmag_Earth, 'LineWidth', 1.5); hold on;
plot(period_percent, vmag_Phaethon, 'LineWidth', 1.5);
grid on;
xlabel('Earth orbital period (%));
ylabel('|v| (km/s));
title('Velocity Magnitude over One Earth Orbit');
legend('Earth', 'Phaethon', 'Location', 'best');
%% Task 1.1.5: Separation between Earth and Phaethon over 25 years

% Time Span: 25 years from t0
years_span = 25;
days_span = years_span * 365.25; % converting to days and counting for leap year

% Time vector in Julian Days
N_sep = 5000; % enough points for a smooth long term plot
t_sep = linspace(t0, t0 + days_span, N_sep);

% Preallocate separation array
sep = zeros(1, N_sep);

% Calculate separation between Earth and Phaethon
for k = 1:N_sep
    X_Earth_sep = Ephemeris(t_sep(k), ObjData1, Sun_mu);
    X_Phaethon_sep = Ephemeris(t_sep(k), ObjData2, Sun_mu);

    sep(k) = norm(X_Earth_sep(1:3) - X_Phaethon_sep(1:3)); % separation in km
end

% Time axis in years from t0
time_years = (t_sep - t0) / 365.25;

% Plot separation over time
figure;
plot(time_years, sep/1e6, 'LineWidth', 1.5);
grid on;
xlabel('Time since t_0 (years));
ylabel('Separation distance (10^6 km));
title('Earth-Phaethon Separation over 25 Years');

%% Numerical Integration
%% Task 1.2.3: Numerical Integration using ode45

% Initial ECI state vector
X0_J2 = [4674.87574558088;
    957.439085778802;
    4668.33830493665;
    -5.02475980211043;
    -2.71977766681309;
    5.58960115240514];

% Extract initial position and velocity
r0_vec = X0_J2(1:3);
v0_vec = X0_J2(4:6);

r0 = norm(r0_vec);
v0 = norm(v0_vec);

% Compute semi-major axis from vis-viva equation
a0 = 1 / ((2/r0) - (v0^2/Earth_mu));

% Orbital period
T_orbit = 2*pi*sqrt(a0^3/Earth_mu);

% Integration time: 15 orbital periods, output every 60 seconds
dt_out = 60;
tspan_J2 = 0:dt_out:(15*T_orbit);

```

```

% ODE45 tolerances required by assignment
options = odeset('RelTol', 1e-12, 'AbsTol', 1e-12);

% Integrate equations of motion with J2 perturbation
[tout_J2, yout_J2] = ode45(@(t,X) TBPwJ2_ECI(t, X, Earth_mu, J2_Earth, radius_Earth), ...
    tspan_J2, X0_J2, options);

% Display useful checks
disp('Initial orbit semi-major axis (km):')
disp(a0)

disp('Initial orbital period (s):')
disp(T_orbit)

disp('Initial orbital period (hours):')
disp(T_orbit/3600)

% Plot 3D trajectory with Earth sphere
figure;
hold on;
grid on;

% Plot spacecraft trajectory
plot3(yout_J2(:,1), yout_J2(:,2), yout_J2(:,3), 'LineWidth', 1.2);

% Create Earth sphere
[xe, ye, ze] = sphere(80);

surf(radius_Earth*xe, radius_Earth*ye, radius_Earth*ze, ...
    'FaceAlpha', 0.3, ...
    'EdgeColor', 'none');

% Labels and formatting
xlabel('x (km)');
ylabel('y (km)');
zlabel('z (km)');
title('ECI Trajectory with J2 Perturbation over 15 Orbital Periods');
legend('Spacecraft trajectory', 'Earth', 'Location', 'best');

axis equal;
view(3);
hold off;

%% Task 1.2.4: Specific Energy and Angular Momentum Analysis

% Extract position and velocity vectors
r_vec = yout_J2(:,1:3);    % km
v_vec = yout_J2(:,4:6);    % km/s

% Magnitudes
r_mag = vecnorm(r_vec, 2, 2);    % km
v_mag = vecnorm(v_vec, 2, 2);    % km/s

% Time in hours
time_hours = tout_J2 / 3600;

% Specific energies (km^2/s^2)
KE_specific = 0.5 * v_mag.^2;
PE_specific = -Earth_mu ./ r_mag;
E_total_specific = KE_specific + PE_specific;

% Specific angular momentum magnitude (km^2/s)
h_vec = cross(r_vec, v_vec, 2);
h_mag = vecnorm(h_vec, 2, 2);

% Plot specific energies
figure;
plot(time_hours, KE_specific, 'LineWidth', 1.3);
hold on;

```

```

plot(time_hours, PE_specific, 'LineWidth', 1.3);
plot(time_hours, E_total_specific, 'LineWidth', 1.5);
grid on;

xlabel('Time (hours)');
ylabel('Specific Energy (km^2/s^2)');
title('Specific Energy Variation over 15 Orbital Periods');
legend('Kinetic Energy', 'Potential Energy', 'Total Energy', 'Location', 'best');

hold off;

% Plot specific angular momentum
figure;
plot(time_hours, h_mag, 'LineWidth', 1.5);
grid on;

xlabel('Time (hours)');
ylabel('Specific Angular Momentum (km^2/s)');
title('Specific Angular Momentum Variation over 15 Orbital Periods');

%% Task 1.2.5: Convert initial ECI state to COE
X0_J2 = [4674.87574558088;
          957.439085778802;
          4668.33830493665;
          -5.02475980211043;
          -2.71977766681309;
          5.58960115240514];

coe_initial = RV2COE(X0_J2, Earth_mu);

% Convert angular elements to degrees for reporting
coe_initial_deg = coe_initial;
coe_initial_deg(3:6) = rad2deg(coe_initial(3:6));

disp('Initial COE state [a (km), e, i (deg), Omega (deg), w (deg), theta (deg)]:')
disp(coe_initial_deg)

%% Task 1.2.6 - Semi-major axis and RAAN evolution
N_J2 = length(tout_J2);

a_history = zeros(N_J2, 1);
Omega_history = zeros(N_J2, 1);

for k = 1:N_J2
    Xk = yout_J2(k, :); % convert row to column vector
    coe_k = RV2COE(Xk, Earth_mu);

    a_history(k) = coe_k(1); % km
    Omega_history(k) = coe_k(4); % rad
end

% Convert RAAN to degrees
Omega_history_deg = rad2deg(Omega_history);

% Unwrap RAAN to show continuous secular drift
Omega_unwrapped_deg = rad2deg(unwrap(Omega_history));

% Time in hours and days
time_hours = tout_J2 / 3600;
time_days = tout_J2 / 86400;

% Plot semi-major axis
figure;
plot(time_hours, a_history, 'LineWidth', 1.5);
grid on;
xlabel('Time (hours)');
ylabel('Semi-major axis, a (km)');

```

```

title('Semi-major Axis Variation under J2 Perturbation');

% Plot RAAN
figure;
plot(time_hours, Omega_unwrapped_deg, 'LineWidth', 1.5);
grid on;
xlabel('Time (hours)');
ylabel('RAAN, \Omega (deg)');
title('RAAN Variation under J2 Perturbation');

% RAAN after 1 day
target_time = 86400; % seconds

[~, idx_1day] = min(abs(tout_J2 - target_time));

Omega_1day_deg = Omega_unwrapped_deg(idx_1day);

disp('RAAN after 1 day (deg):')
disp(Omega_1day_deg)

disp('Initial RAAN (deg):')
disp(Omega_unwrapped_deg(1))

disp('RAAN change after 1 day (deg):')
disp(Omega_1day_deg - Omega_unwrapped_deg(1))

%% Orbital Maneuvers and Mission Design
%% Geostationary Orbit insertion
%% Task 2.1.3 - COE evolution during multi-burn GEO insertion

muE = 3.986e5;    % km^3/s^2
RE = 6378.14;    % km
rGEO = 42164;    % km
i0 = deg2rad(28.5); % rad

% GTO parameters
rp = RE + 200;    % km
ra = rGEO;        % km
aGTO = (rp + ra)/2;

% Speeds at GEO radius
vGTO_a = sqrt(muE*(2/ra - 1/aGTO));
vGEO = sqrt(muE/rGEO);

% Initial apogee state
r_vec = [rGEO; 0; 0];

v_GTO = [0;
    vGTO_a*cos(i0);
    vGTO_a*sin(i0)];

v_GEO = [0;
    vGEO;
    0];

% Total insertion delta-v
dv_total = v_GEO - v_GTO;
dv_total_mag = norm(dv_total);

% Burn constraint
F = 1400;    % N
msat = 1000; % kg
tthr = 400;  % s

dv_max = (F/msat)*tthr/1000; % km/s
Nman = ceil(dv_total_mag/dv_max);

% Identical burn increment

```

```

dv_n = dv_total/Nman;

% Store COE history
coe_history = zeros(Nman+1, 6);

% Initial state before burns
X = [r_vec; v_GTO];
coe_history(1,:) = RV2COE(X, muE);

% Apply burns
for n = 1:Nman
    X(4:6) = X(4:6) + dv_n;
    coe_history(n+1,:) = RV2COE(X, muE);
end

% Convert angles from radians to degrees
coe_table_data = coe_history;
coe_table_data(:,3:6) = rad2deg(coe_table_data(:,3:6));

% Create table
Burn = (0:Nman)';

COE_Table = table(Burn, ...
    coe_table_data(:,1), ...
    coe_table_data(:,2), ...
    coe_table_data(:,3), ...
    coe_table_data(:,4), ...
    coe_table_data(:,5), ...
    coe_table_data(:,6), ...
    'VariableNames', {'Burn_n', 'a_km', 'e', 'i_deg', 'RAAN_deg', 'omega_deg', 'theta_deg'});

disp(COE_Table);

%% Task 2.1.4 Time Analysis of GEO Transfer

% Compute orbital periods
T = 2*pi*sqrt(coe_history(:,1).^3 / muE);

T_hours = T / 3600;
T_days = T / 86400;

% Add to table
COE_Table.T_hours = T_hours;
COE_Table.T_days = T_days;

% Compute total time (exclude final orbit)
t_total_sec = sum(T(1:end-1));
t_total_days = t_total_sec / 86400;

fprintf('Total elapsed time = %.3f days\n', t_total_days);

% Display updated table
disp(COE_Table);

fprintf('\nTotal insertion delta-v = %.6f km/s\n', dv_total_mag);
fprintf('Maximum delta-v per burn = %.6f km/s\n', dv_max);
fprintf('Number of burns = %d\n', Nman);

%% Phasing Maneuvers

%% Task 2.2.3 - Phasing orbit elements and delta-v

% Constants
muE = 3.986e5;    % km^3/s^2
rGEO = 42164;    % km
TGEO = 86164;    % s
delta_lambda = -80; % deg

% From Task 2.2.2

```

```

dt_ph = (delta_lambda/360)*TGEO;
Tph = TGEO - dt_ph;

% Phasing orbit semi-major axis
aph = (muE*(Tph/(2*pi))^2)^(1/3);

% External phasing orbit: perigee at GEO
rp_ph = rGEO;
ra_ph = 2*aph - rp_ph;
eph = (ra_ph - rp_ph)/(ra_ph + rp_ph);

% Velocities
vGEO = sqrt(muE/rGEO);
vph_p = sqrt(muE*(2/rGEO - 1/aph));

% Delta-v
dv1 = vph_p - vGEO;
dv2 = vGEO - vph_p;
dv_total = abs(dv1) + abs(dv2);

% Display results
fprintf('Tph = %.3f s = %.3f hours\n', Tph, Tph/3600);
fprintf('aph = %.3f km\n', aph);
fprintf('rp_ph = %.3f km\n', rp_ph);
fprintf('ra_ph = %.3f km\n', ra_ph);
fprintf('eph = %.6f\n', eph);
fprintf('vGEO = %.6f km/s\n', vGEO);
fprintf('vph_p = %.6f km/s\n', vph_p);
fprintf('dv1 = %.6f km/s\n', dv1);
fprintf('dv2 = %.6f km/s\n', dv2);
fprintf('Total phasing delta-v = %.6f km/s\n', dv_total);

% Plot GEO and phasing orbit
theta = linspace(0, 2*pi, 1000);

% GEO orbit
xGEO = rGEO*cos(theta);
yGEO = rGEO*sin(theta);

% Phasing orbit in polar form, with perigee at theta = 0
p_ph = aph*(1 - eph^2);
r_ph = p_ph ./ (1 + eph*cos(theta));

xPH = r_ph.*cos(theta);
yPH = r_ph.*sin(theta);

figure; hold on; grid on; axis equal;

% Earth marker
plot(0, 0, 'ko', 'MarkerFaceColor', 'k');

% Orbits
plot(xGEO, yGEO, 'b', 'LineWidth', 2);
plot(xPH, yPH, 'r--', 'LineWidth', 2);

% Mark burn point at GEO/perigee of phasing orbit
plot(rGEO, 0, 'ko', 'MarkerFaceColor', 'y', 'MarkerSize', 7);
text(rGEO, 0, ' GEO / phasing burn point');

legend('Earth', 'GEO orbit', 'External phasing orbit', 'Burn point', ...
    'Location', 'best');

xlabel('x (km)');
ylabel('y (km)');
title('GEO and External Phasing Orbit');

%% Task 2.2.4 - Multi-revolution phasing periods
Nph = (1:20);    % number of phasing revolutions

```

```

dt_ph_N = (delta_lambda./(360*Nph))*TGEO; % s per revolution
Tph_N = TGEO - dt_ph_N; % s
tph_N_days = (Nph.*Tph_N)/86400; % days

PhasingTime_Table = table(Nph, Tph_N, Tph_N/3600, tph_N_days, ...
    'VariableNames', {'Nph','Tph_s','Tph_hours','Total_time_days'});

disp(PhasingTime_Table);

%% Task 2.2.5 - Phasing orbit size and delta-v optimisation

% Semi-major axis for each phasing case
aph_N = (muE*(Tph_N/(2*pi)).^2).^(1/3);

% GEO and phasing-orbit perigee velocities
vGEO = sqrt(muE/rGEO);
vph_p_N = sqrt(muE*(2/rGEO - 1./aph_N));

% Total delta-v for two tangential burns
dv_ph_N = 2*abs(vph_p_N - vGEO);

% Offset from GEO radius
aph_offset = aph_N - rGEO;

% Add results to table
PhasingTime_Table.a_ph_km = aph_N;
PhasingTime_Table.a_ph_minus_rGEO_km = aph_offset;
PhasingTime_Table.dv_ph_kms = dv_ph_N;

disp(PhasingTime_Table);

% Apply 10-day mission constraint
valid_idx = tph_N_days <= 10;

valid_Nph = Nph(valid_idx);
valid_dv = dv_ph_N(valid_idx);

[best_dv, best_local_idx] = min(valid_dv);
best_Nph = valid_Nph(best_local_idx);
best_time = tph_N_days(Nph == best_Nph);
best_aph = aph_N(Nph == best_Nph);

fprintf('\nBest Nph under 10 days = %d\n', best_Nph);
fprintf('Minimum delta-v = %.6f km/s\n', best_dv);
fprintf('Total phasing time = %.3f days\n', best_time);
fprintf('Phasing semi-major axis = %.3f km\n', best_aph);

% Plot a_ph - rGEO against total phasing time

figure; hold on; grid on;
plot(tph_N_days, aph_offset, 'o-', 'LineWidth', 1.5);
xline(10, '--', '10 day limit');

xlabel('Total phasing time, t_{ph,N} (days)');
ylabel('a_{ph,N} - r_{GEO} (km)');
title('Phasing Orbit Size vs Total Phasing Time');

% Plot delta-v against total phasing time

figure; hold on; grid on;
plot(tph_N_days, dv_ph_N, 'o-', 'LineWidth', 1.5);
xline(10, '--', '10 day limit');
plot(best_time, best_dv, 'kp', 'MarkerFaceColor', 'y', 'MarkerSize', 10);

xlabel('Total phasing time, t_{ph,N} (days)');
ylabel('\Delta v_{ph,N} (km/s)');
title('Phasing Delta-v vs Total Phasing Time');

```

```

legend('Phasing cases', '10 day limit', 'Best feasible case', ...
      'Location', 'best');

```

Figure 21: Main Script

```

function[theta] = Kepler(e,M,Tol)
% Kepler's Equation Solver using Newton-Raphson
% Inputs:
% e - eccentricity (0 <= e < 1)
% M - mean anomaly (radians)
% Tol - tolerance for convergence
%
% Output:
% theta - true anomaly (radians)

% Initial Guess for E
E = M; % Letting the eccentricity anomaly equal the mean anomaly
% Newton-Raphson iteration
error = 1;
while error > Tol
    f = E - e*sin(E) - M;    % Kepler's equation
    f_prime = 1 - e*cos(E);  % first derivative

    E_new = E - f / f_prime;  % Newton-Raphson update

    error = abs(E_new - E);    % Checking estimation change
    if abs(E_new - E) < Tol && abs(f) < Tol % Ensure convergence by checking both the update step (stability) and the residual |f(E)|
        break;
    end
    E = E_new;                % Update Eccentricity anomaly
end
% Convert eccentric anomaly E to true anomaly theta
theta = 2 * atan2( sqrt(1+e)*sin(E/2), sqrt(1-e)*cos(E/2) );
end

```

Figure 22: Kepler Function

```

function[X] = COE2RV(oe,mu)
% COE2RV Converts classical orbital elements to inertial position and velocity
%
% Inputs:
% oe = [a; e; i; Omega; w; theta]
% a - semi-major axis (km)
% e - eccentricity (-)
% i - inclination (rad)
% Omega - RAAN (rad)
% w - argument of periapsis (rad)
% theta - true anomaly (rad)
% mu - gravitational parameter of central body (km^3/s^2)
%
% Output:
% X = [x; y; z; vx; vy; vz] in inertial frame

a = oe(1);
e = oe(2);
i = oe(3);
Omega = oe(4);
w = oe(5);
theta = oe(6);

% Semi-latus rectum
p = a * (1-e^2);

% Position and Velocity in the perifocal frame
r_PQW = (p / (1 + e * cos(theta))) * [cos(theta); sin(theta); 0];
v_PQW = sqrt(mu/p) * [-sin(theta); e + cos(theta); 0];

% Rotation matrix from PQW to inertial frame

```

```

Q = [
    cos(Omega)*cos(w) - sin(Omega)*sin(w)*cos(i), -cos(Omega)*sin(w) - sin(Omega)*cos(w)*cos(i), sin(Omega)*sin(i);
    sin(Omega)*cos(w) + cos(Omega)*sin(w)*cos(i), -sin(Omega)*sin(w) + cos(Omega)*cos(w)*cos(i), -cos(Omega)*sin(i);
    sin(w)*sin(i), cos(w)*sin(i), cos(i)
];

% Converting to inertial frame
r_IJK = Q * r_PQW;
v_IJK = Q * v_PQW;

% State vector
X = [r_IJK; v_IJK];
end

```

Figure 23: COE2RV Function

```

function X = Ephemeris(t, ObjData, mu)
% Ephemeris returns inertial position and velocity at time t
%
% Inputs:
% t - time in Julian Days
% ObjData - [t0; a; e; i; Omega; w; M0]
% t0 : reference epoch (Julian Days)
% a : semi-major axis (km)
% e : eccentricity (-)
% i : inclination (deg)
% Omega : RAAN (deg)
% w : argument of periapsis (deg)
% M0 : mean anomaly at t0 (deg)
% mu - gravitational parameter of central body (km^3/s^2)
%
% Output:
% X = [x; y; z; vx; vy; vz] inertial state vector

Tol = 1e-12;

% Extract data
t0 = ObjData(1);
a = ObjData(2);
e = ObjData(3);
i = deg2rad(ObjData(4));
Omega = deg2rad(ObjData(5));
w = deg2rad(ObjData(6));
M0 = deg2rad(ObjData(7));

% Elapsed time in seconds
dt = (t - t0) * 24 * 3600;

% Mean motion
n = sqrt(mu / a^3);

% Propagate mean anomaly
M = M0 + n * dt;
M = mod(M, 2*pi);

% Solve for true anomaly
theta = Kepler(e, M, Tol);

% Updated orbital elements
oe = [a; e; i; Omega; w; theta];

% Convert to inertial state
X = COE2RV(oe, mu);
end

```

Figure 24: Ephemeris Function

```
function f = TBPwJ2_ECI(t, X, mu, J2, RE)
% TBPwJ2_ECI
% Returns the time derivative of the ECI state vector for the
% two-body problem including the J2 perturbation.
%
% Inputs:
% t - time (s) [not used explicitly, but required by ode45]
% X - state vector [x; y; z; vx; vy; vz] in km and km/s
% mu - gravitational parameter of central body (km^3/s^2)
% J2 - second zonal harmonic (-)
% RE - equatorial radius of central body (km)
%
% Output:
% f - state derivative [vx; vy; vz; ax; ay; az]

% Extract position and velocity
x = X(1);
y = X(2);
z = X(3);
vx = X(4);
vy = X(5);
vz = X(6);

% Radius magnitude
r = sqrt(x^2 + y^2 + z^2);

% Two-body acceleration
ax_2B = -mu*x/r^3;
ay_2B = -mu*y/r^3;
az_2B = -mu*z/r^3;

% J2 acceleration
factor_J2 = -(3*mu*J2*RE^2)/(2*r^5);

ax_J2 = factor_J2 * x * (1 - 5*z^2/r^2);
ay_J2 = factor_J2 * y * (1 - 5*z^2/r^2);
az_J2 = factor_J2 * z * (3 - 5*z^2/r^2);

% Total acceleration
ax = ax_2B + ax_J2;
ay = ay_2B + ay_J2;
az = az_2B + az_J2;

% State derivative
f = [vx;
    vy;
    vz;
    ax;
    ay;
    az];
end
```

Figure 25: TBwJ2_ECI Function

```
function coe = RV2COE(X, mu)
% RV2COE Converts ECI position and velocity to classical orbital elements
%
% Inputs:
% X = [x; y; z; vx; vy; vz] (km, km/s)
% mu = gravitational parameter (km^3/s^2)
%
% Output:
% coe = [a; e; i; Omega; w; theta]
% angles returned in radians

r_vec = X(1:3);
```

```

v_vec = X(4:6);

r = norm(r_vec);
v = norm(v_vec);

% Angular momentum
h_vec = cross(r_vec, v_vec);
h = norm(h_vec);

% Node vector
k_hat = [0; 0; 1];
n_vec = cross(k_hat, h_vec);
n = norm(n_vec);

% Eccentricity vector
e_vec = (1/mu) * ((v^2 - mu/r)*r_vec - dot(r_vec, v_vec)*v_vec);
e = norm(e_vec);

% Specific mechanical energy
energy = 0.5*v^2 - mu/r;

% Semi-major axis
a = -mu / (2*energy);

% Inclination
i = acos(h_vec(3)/h);

% RAAN
Omega = atan2(n_vec(2), n_vec(1));
Omega = mod(Omega, 2*pi);

% Argument of periapsis
w = atan2(dot(cross(n_vec, e_vec), h_vec)/h, dot(n_vec, e_vec));
w = mod(w, 2*pi);

% True anomaly
theta = atan2(dot(cross(e_vec, r_vec), h_vec)/h, dot(e_vec, r_vec));
theta = mod(theta, 2*pi);

% COE vector
coe = [a; e; i; Omega; w; theta];
end

```

Figure 26: RV2COE Function